

# Autonomous Driving Debugging Special Topic

---

## Autonomous Driving Debugging Special Topic

1. Course Content
2. YOLOv11 Inference Parameter Debugging (Not applicable to RDK)
  - 2.1 Traffic Sign Recognition Debugging
  - 2.2 Parameter Adjustment Guide
  - 2.3 Lane Line Detection Debugging
  - 2.4 Parameter Adjustment Guide
3. Lane Left-Right Keeping—Zero Offset Error Calibration
4. Lane Keeping Left-Right Keeping—PID Debugging
  - 4.1 Parameter Adjustment Guide:
5. Lane Straight Keeping—PID Debugging
  - 5.1 Parameter Adjustment Guide:
6. Curve Turning Effect Debugging
  - 6.1 Parameter Adjustment Guide:
7. Right Turn Sign Debugging
8. Parking Sign Debugging

## 1. Course Content

---

- Master the role and debugging methods of autonomous driving YOLO inference recognition, lane centering keeping, and traffic sign recognition response parameters to optimize autonomous driving effects.

## 2. YOLOv11 Inference Parameter Debugging (Not applicable to RDK)

---

[!CAUTION]

**RDK mainboard users cannot debug inference parameters because the inference model is a quantized model. Use factory preset parameters!!**

- YOLOv11 parameter file location:

```
~/yahboomM3_ws/user_ws/src/m3_bringup/config/auto_drive/yolo_param.yaml
```

- Parameter content is as follows, where
  - The `sign_detection` field contains inference parameters for the traffic sign recognition model
  - The `line_detection` field contains inference parameters for the lane line detection model

```
#Road sign detection inference parameters
sign_detection:
  model_path: '/home/jetson/MODELS/auto_drive/sign_model.engine'
```

conf: 0.92

#Sets the minimum confidence threshold for detection. Objects detected with a confidence level below this threshold will be ignored. Adjusting this value can help reduce false positives.

iou: 0.7

#An overlap threshold is used for non-maximum suppression (NMS). Lower values reduce detections by eliminating overlapping bounding boxes.

rect: True

#If enabled, the shorter side of the image is minimally padded until it is divisible by the step size to improve inference speed.

half: True

#If enabled, half-precision inference is used, which can speed up model inference on supported GPUs with minimal impact on accuracy.

device: "cuda:0"

#Reasoning device

batch : 50

#Batch size: Larger batch sizes can provide higher throughput, thereby reducing the total time required for inference.

max\_det : 2

#Maximum number of objects detected

augment : False

#Enabling Test-Time Enhancement (TTA) for prediction may improve the robustness of detection, but it will reduce inference speed.

verbose : False

#Whether to output detailed inference information to the terminal.

classes : [0, 1, 2, 3, 4, 5, 6, 8, 9, 10, 11, 13]

line\_detection:

#Lane detection inference parameters

model\_path: '/home/jetson/MODELS/auto\_drive/lane\_v6\_int8.engine'

conf: 0.4

iou: 0.7

rect: True

half: False

device: "cuda:0"

batch : 50

max\_det : 5

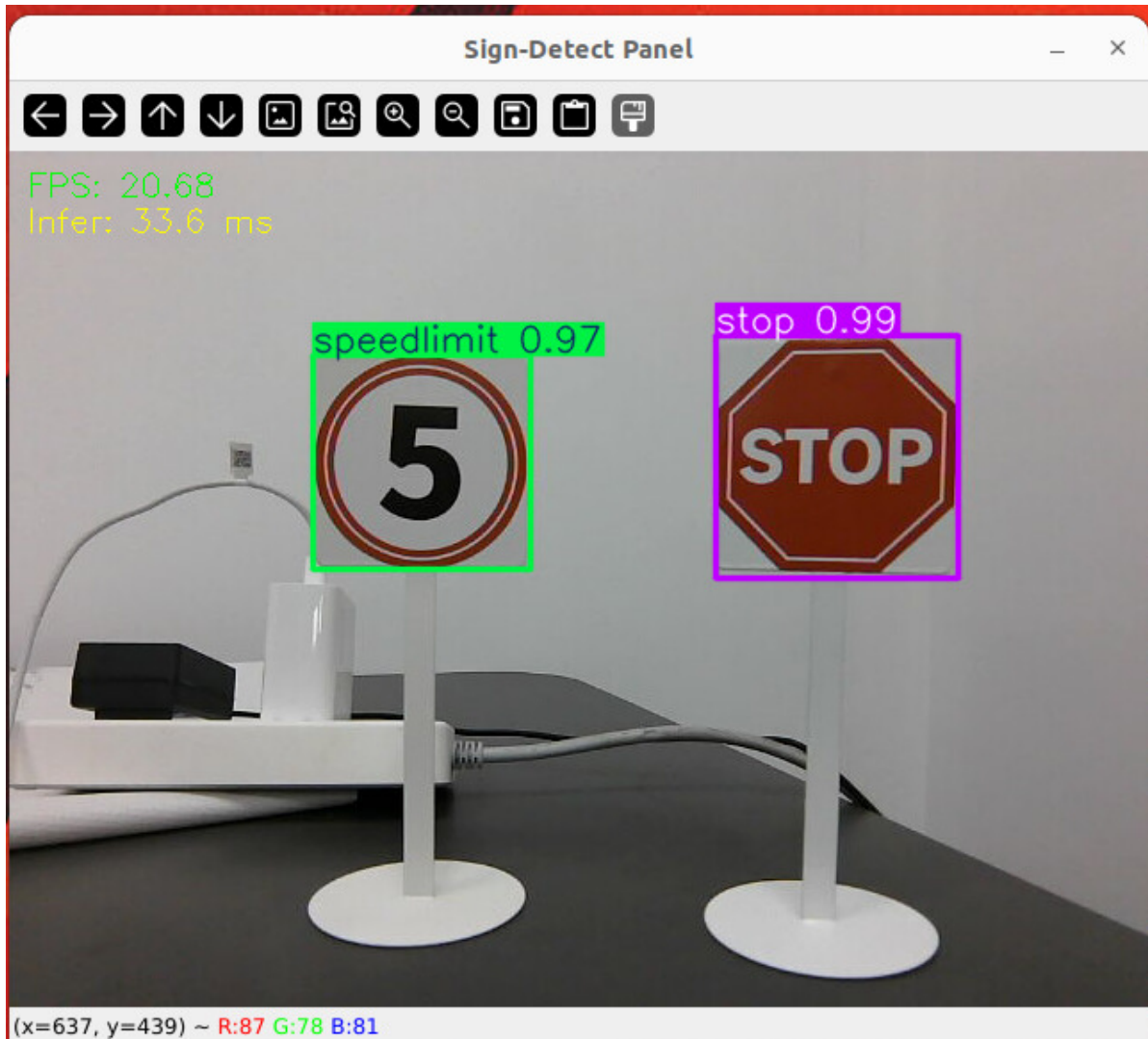
augment : False

verbose : False

## 2.1 Traffic Sign Recognition Debugging

- Start traffic sign recognition program

```
ros2 launch m3_bringup auto_drive.launch.py unkeeplane:=True
```

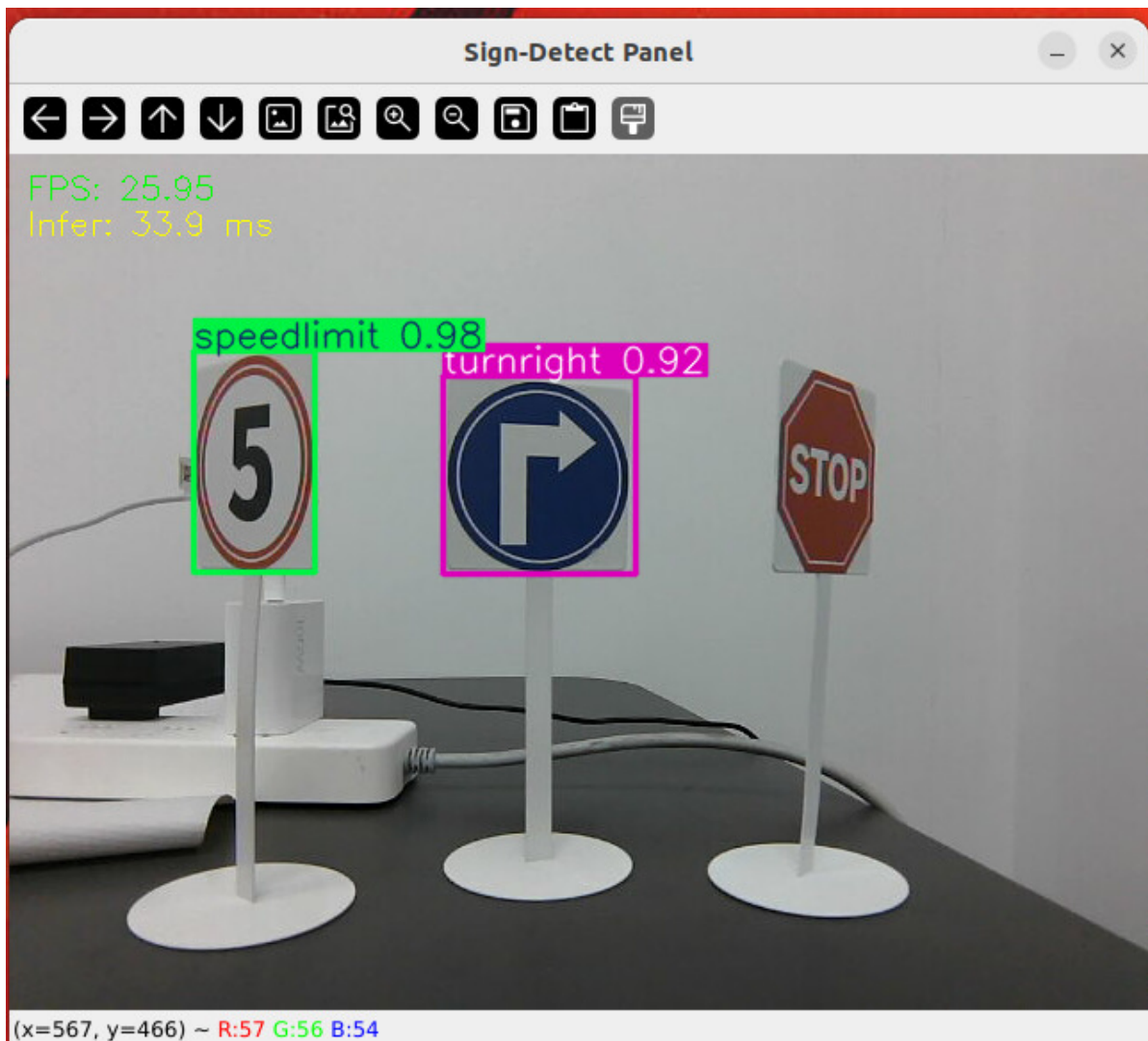


[!TIP]

- Different types of targets are distinguished by different detection boxes. The detection box will display the target type name and confidence level. The target type name here is determined by the data annotation and training stages.

## 2.2 Parameter Adjustment Guide

- If target recognition is not sensitive, for example, the `STOP` sign is not easily recognized from a side view, refer to the following methods:
  - You can reduce the `conf` minimum confidence threshold in the `sign_detection` field
  - Set `augment : True` in the `sign_detection` field to enable enhancement (TTA) for prediction (not recommended).
  - Retrain the model, increase training samples under various angles, lighting, and backgrounds. The more diverse the training samples, the better the model recognition accuracy.



- How to ignore targets during object detection?

If you don't want to detect certain categories of targets, you can adjust the `classes` list in the `sign_detection` field. The numbers in the `classes` list are the targets that need to be detected. If you don't need to detect a certain target, simply remove the corresponding serial number.

```
classes : [0, 1, 2, 3, 4, 5, 6, 8, 9, 10, 11, 13]
```

- Target categories and corresponding serial numbers are shown in the following table

```
#####
#           Traffic sign inspection
#####
# 0: greenlight      绿灯
# 1: honking         鸣笛
# 2: parkA          停车场A
# 3: parkB          停车场B
# 4: redlight       红灯
# 5: school         学校
# 6: sidewalk       人行道
# 7: sidewalk_ground 地面人行道
# 8: speedlimit     限速
# 9: stop           停止
# 10: straight      直行
```

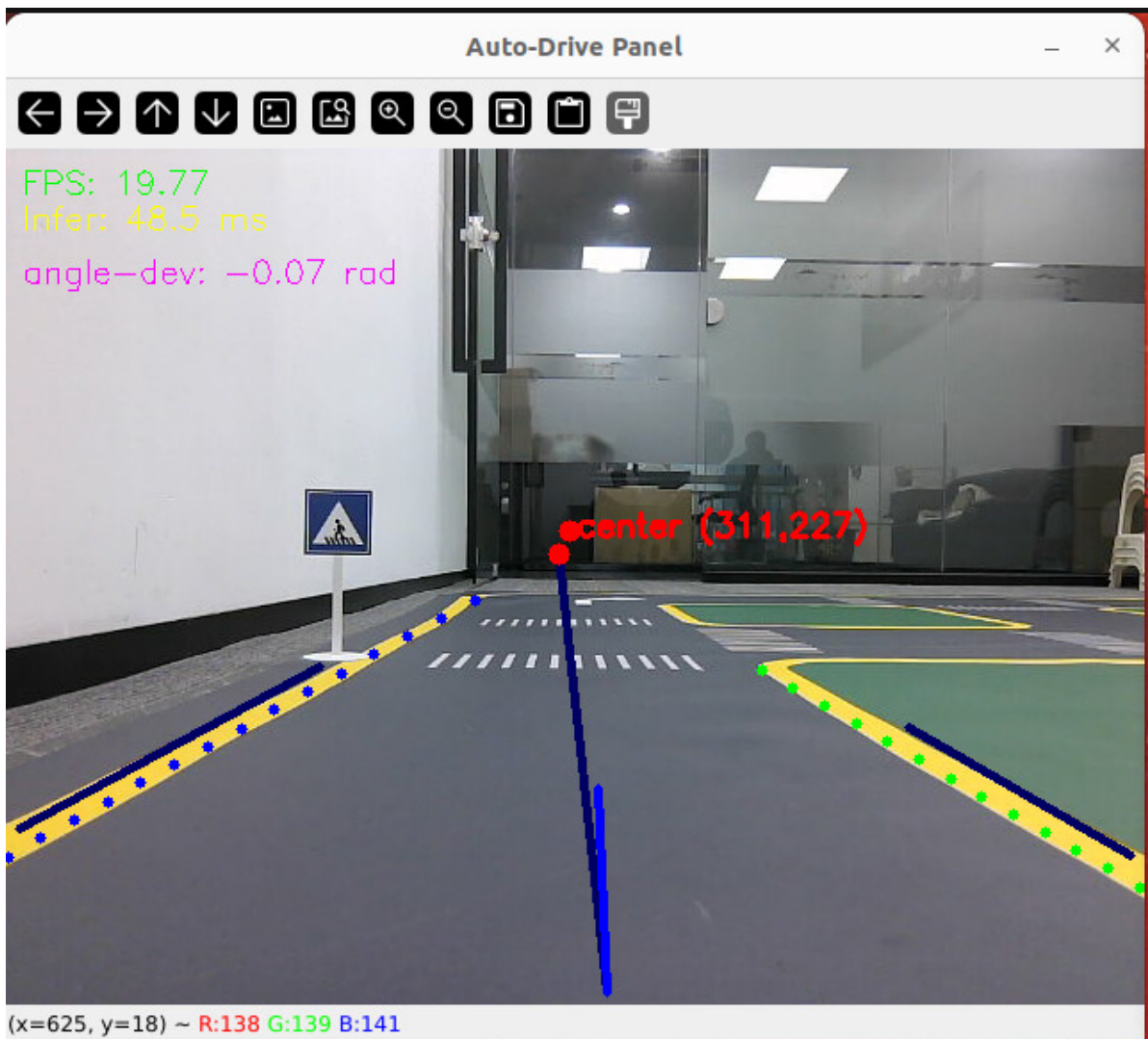


```
# 11: turnright          右转
# 12: turnright_ground  地面右转
# 13: yellowlight       黄灯
#####
#           Lane detection
# 0: lane
#####
```

## 2.3 Lane Line Detection Debugging

- Start autonomous driving program

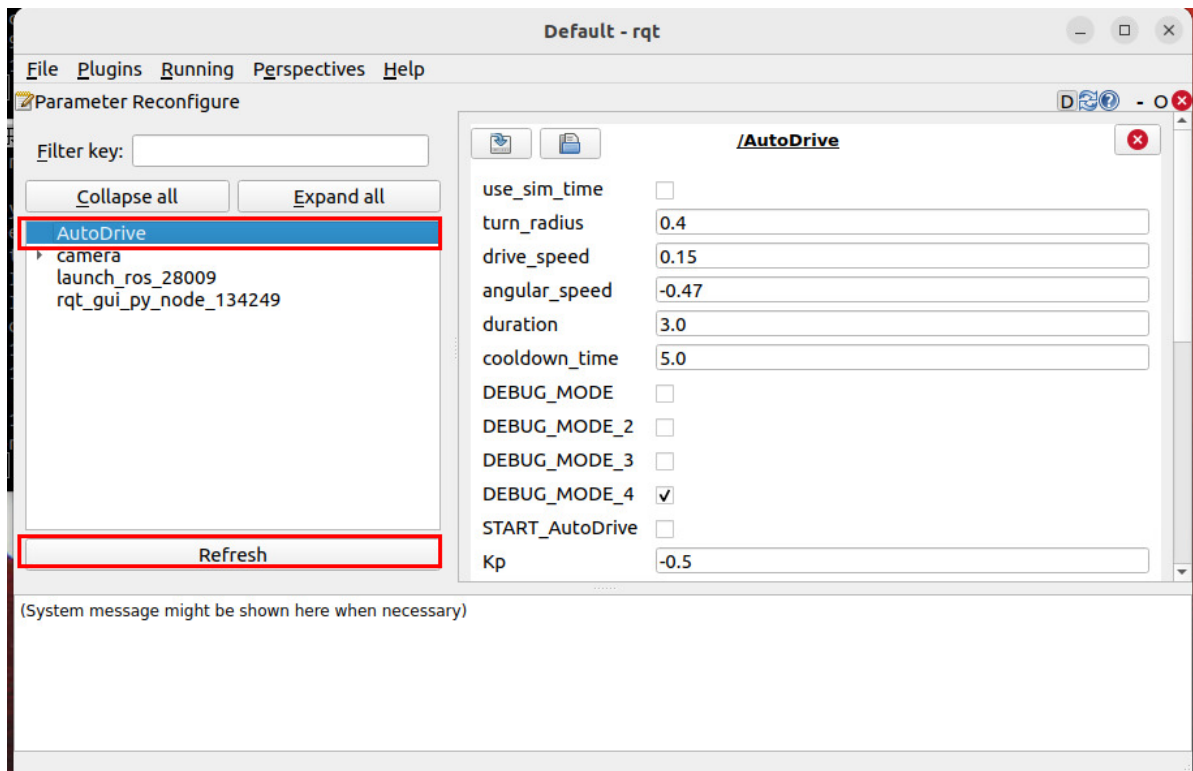
```
ros2 launch m3_bringup auto_drive.launch.py unsign:=True
```



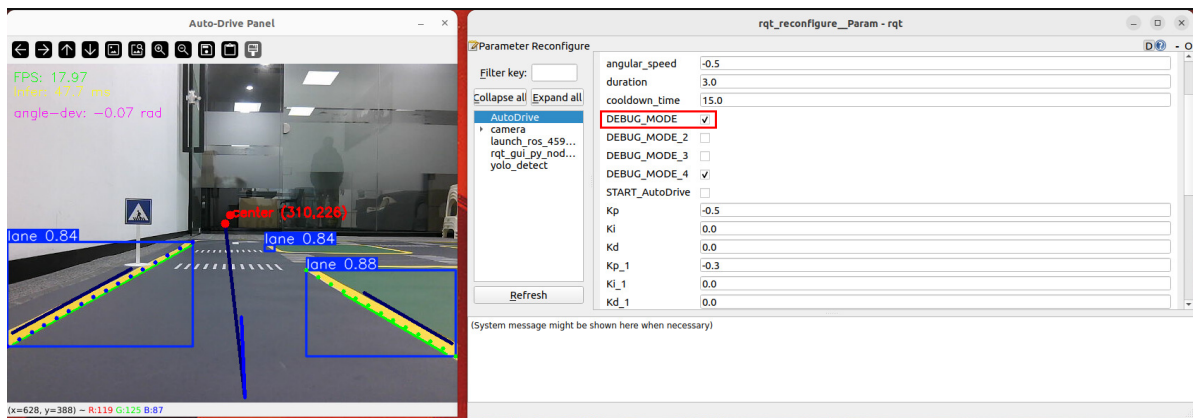
- Start dynamic parameter configuration panel

```
ros2 run rqt_reconfigure rqt_reconfigure
```

- Click `AutoDrive` to enter the parameter control panel. If there is no `AutoDrive` on the left side when you first open it, click `Refresh` below



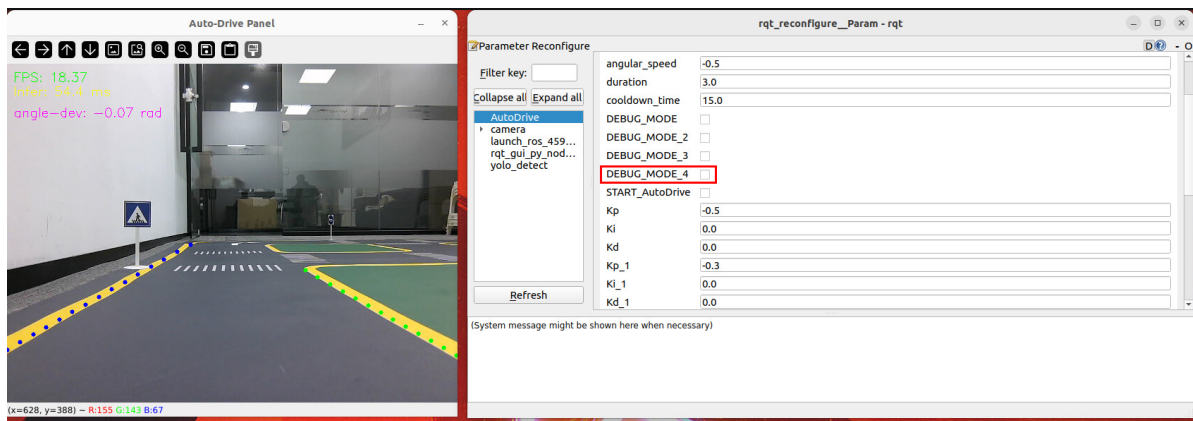
- Click to check `DEBUG_MODE` to display the YOLO detection boxes for lane lines



## 2.4 Parameter Adjustment Guide

[TIP]

- If lane line recognition is not sensitive, refer to traffic sign recognition debugging and reduce the `conf` minimum confidence threshold in the `line_detection` field
- If it is too sensitive and incorrectly shows detection boxes for unrelated lines, increase this detection threshold
- How to close the display of reference lane lines and vehicle body centerline?
- Temporary closure: In the visualization parameter panel, uncheck `DEBUG_MODE_4` to close the display



- Long-term closure: Open the configuration parameter file of the auto\_drive node:

```
~/yahboomM3_ws/user_ws/src/m3_bringup/config/auto_drive/auto_drive_node.yaml
```

- Adjust the `DEBUG_MODE_4` parameter to False and save the file

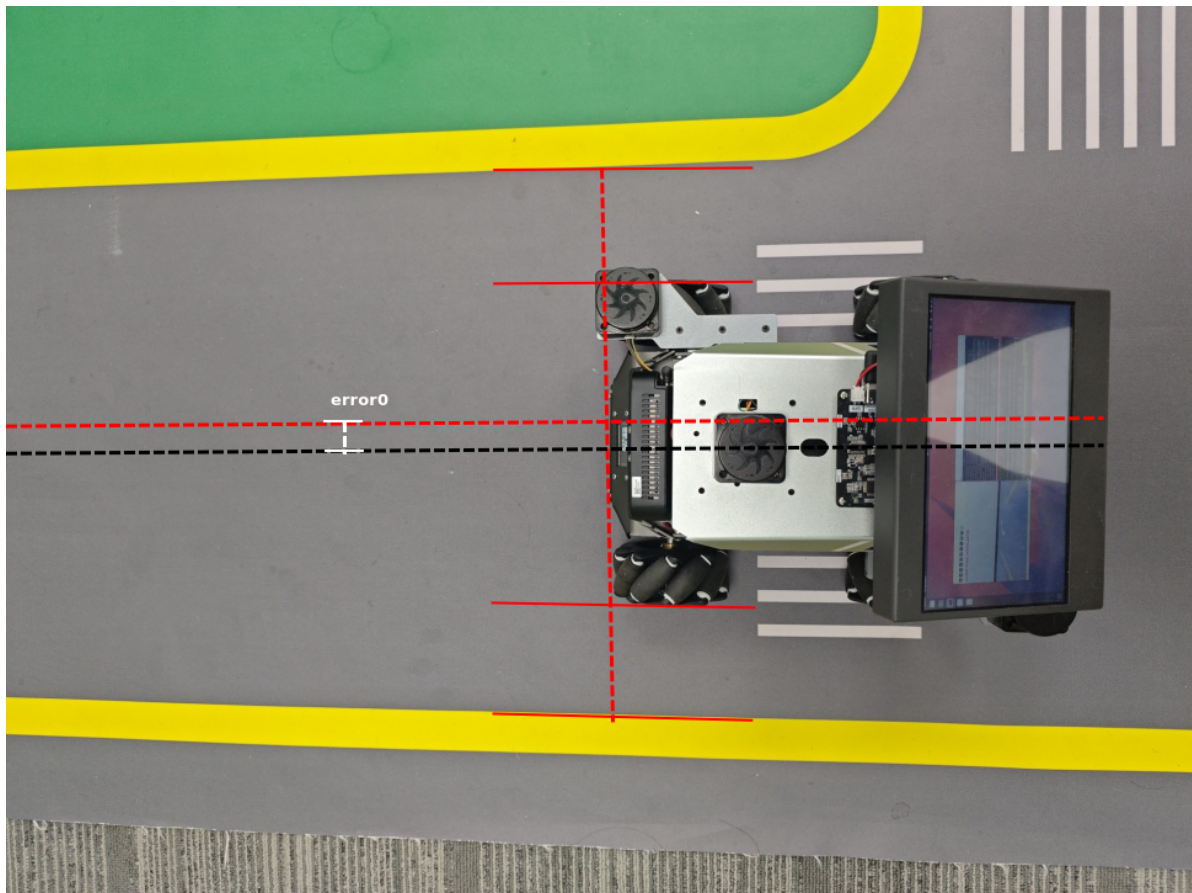
[!NOTE]

The m3\_bringup package uses symbolic link compilation. Parameter modifications will take effect immediately after saving.

```
AutoDrive:
  ros__parameters:
    turn_radius: 0.4          # Turning radius
    drive_speed: 0.15        # Driving speed
    angular_speed: -0.5      # Angular speed
    duration: 3.0            # Turning time
    cooldown_time: 15.0     # Road sign recognition event response time
  interval (seconds)
    DEBUG_MODE: False       #Display the yolo recognition box
    DEBUG_MODE_2: False     #Video debugging mode
    DEBUG_MODE_4: False     #Display the reference lines for the left and
  right lanes
    START_AutoDrive: False
    unkeeplane: False       #Close lane keep driving
    #Linear_PID parameters
    Kp: -0.5
    Ki: 0.0
    Kd: 0.0
    #Lane_PID parameters
    Kp_1: -0.3
    Ki_1: 0.0
    Kd_1: 0.0
    min_error_angular: 0.03 #Maintain the minimum Angle adjustment
  threshold of the straight line
    max_integral: 1000.0    #Integral limit
    image_size: [640, 480]
    combine_nav: False
    roadnet_file: ""
    initial_error: 2.55     #Zero-offset error
    turn_angle: -96.0       # Turning angle
    turn_distance: 0.85     # Turning distance
    park_distance_1: 0.35   # Parking distance
```

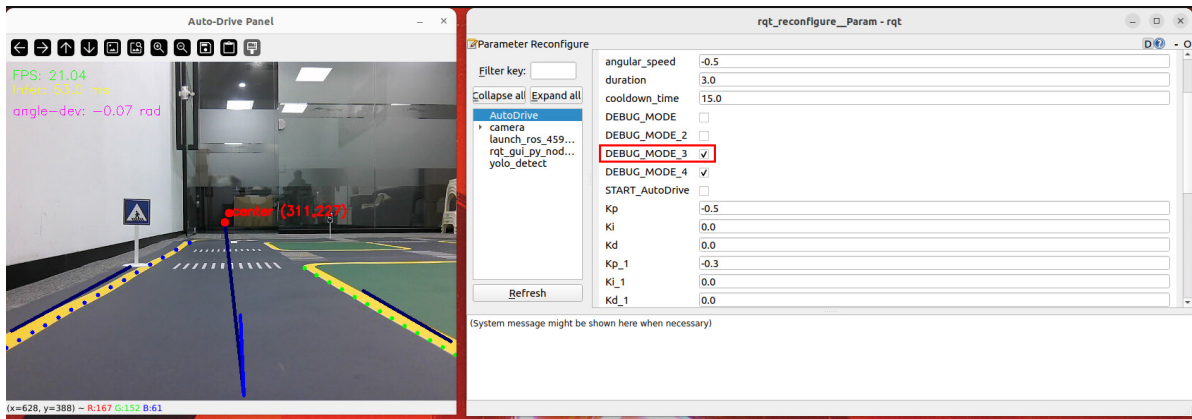
### 3. Lane Left-Right Keeping—Zero Offset Error Calibration

- In the [07.Vehicle Centerline Calibration] course of this chapter, we learned to calibrate the left and right lane reference lines in the camera perspective
- However, when the camera position changes mechanically, or due to small jumps in the YOLO recognition lane line bounding box itself, even if the vehicle body is placed in the left-right center position of the lane, the left-right error may not be zero at this time. This is the zero offset error `error0`. When performing PID operation, this zero offset error needs to be subtracted to obtain the true error value

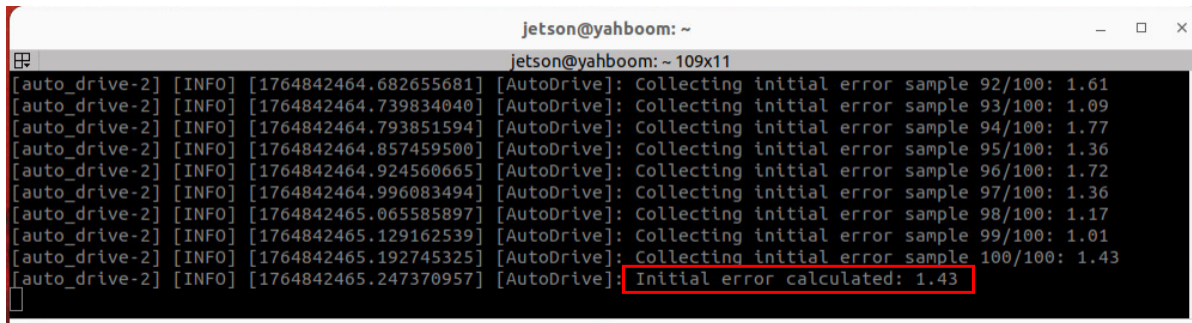


- First place the vehicle body in the center of the lane, click to check `DEBUG_MODE_3`, then it will automatically sample the current left-right deviation error 100 times and calculate the average value as the zero offset error

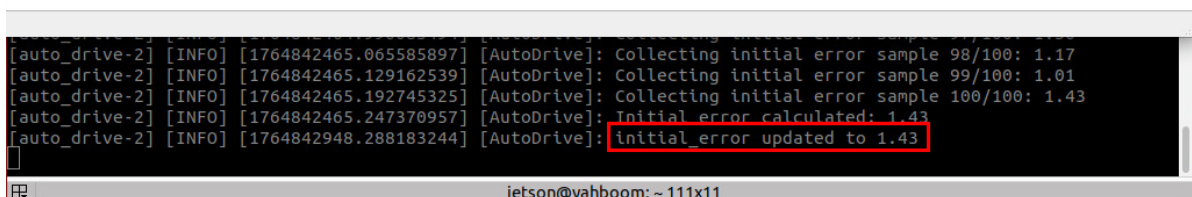
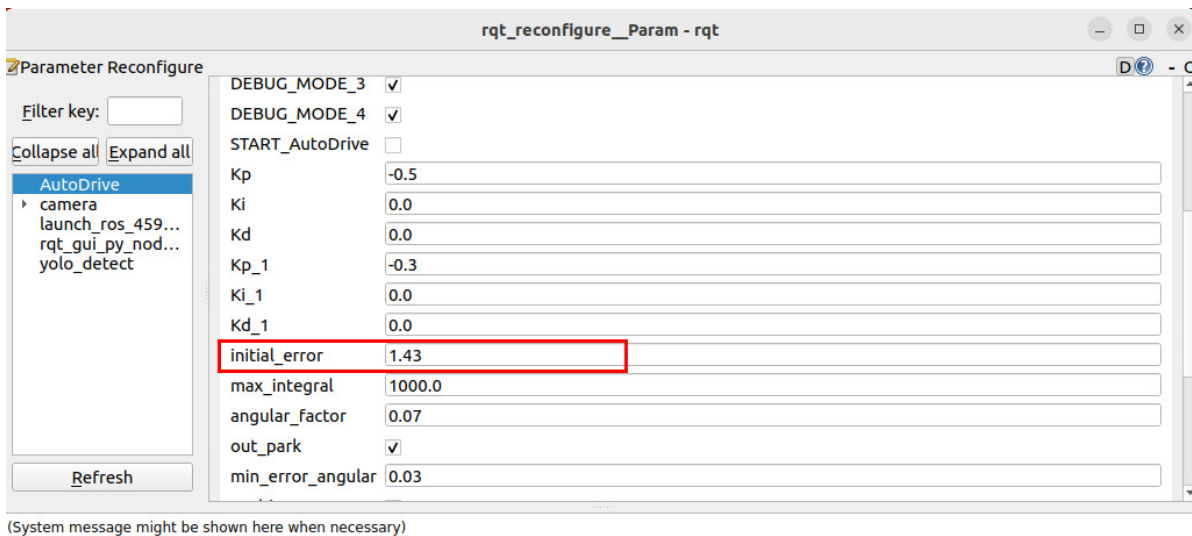




- The terminal will print out the zero offset error value



- How to add zero offset error value to optimize lane left-right centering effect
- Temporary adjustment: Fill in the zero offset error value in init\_error in the rqt\_reconfigure panel, then press Enter, and the terminal will prompt that the parameter has been updated



- Long-term effect:

Open the configuration parameter file of the auto\_drive node, modify initial\_error, and then save.

```

AutoDrive:
  ros__parameters:
    .....
    initial_error: 1.43          #Zero-offset error
    .....

```

## 4. Lane Keeping Left-Right Keeping—PID Debugging

- The vehicle body's left-right keeping in the lane is controlled by PID on the left-right deviation. The control parameters are Kp\_1, Ki\_1, Kd\_1. We can temporarily adjust and observe the actual effect in `rat_reconfigure`

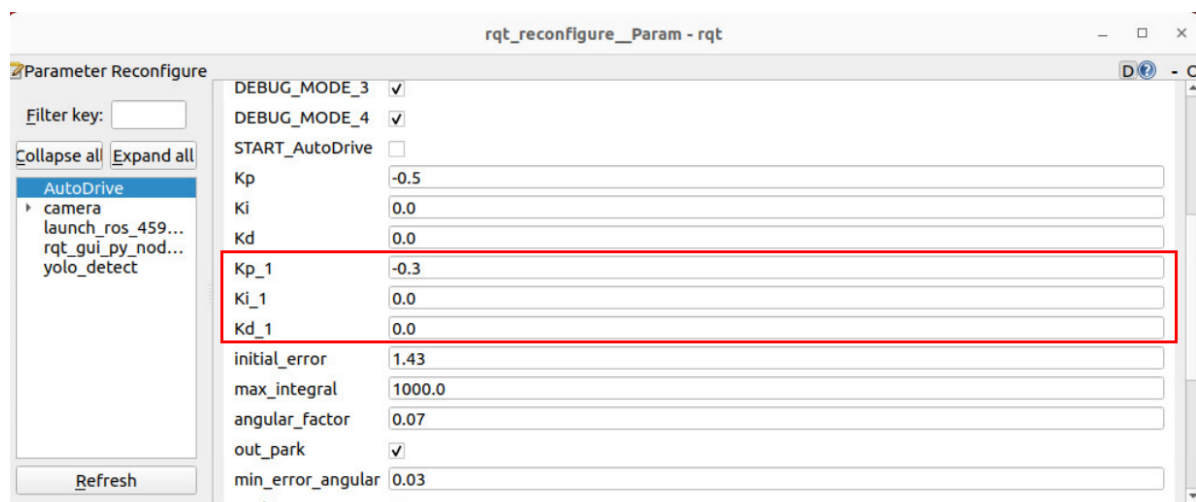
[!WARNING]

**Note:** Lane left-right keeping is not always performed. It is only when the deviation angle between the vehicle body centerline and the lane centerline is less than the `min_error_angular` threshold that left-right adjustment will be performed.

### 4.1 Parameter Adjustment Guide:

[!TIP]

- If the left-right translation centering effect is not obvious or sluggish, increase the absolute value of Kp\_1, with a negative sign
- If left-right translation is frequent and normal Kp\_1 shows overshoot oscillation, you can reduce Kp\_1 or introduce Kd\_1 differential term to increase damping.
- Ki is used to control steady-state error. Here, in the small-scale scenario of the sandbox map, fast response is needed, so the integral term is not introduced.



- After parameter debugging is completed, if you need long-term effective parameters, open the configuration parameter file of the auto\_drive node, modify the corresponding parameters, and then save.

```
AutoDrive:
  ros__parameters:
    #Lane_PID parameters
    Kp_1: -0.3
    Ki_1: 0.0
    Kd_1: 0.0
    max_integral: 1000.0          #Integral limit
```

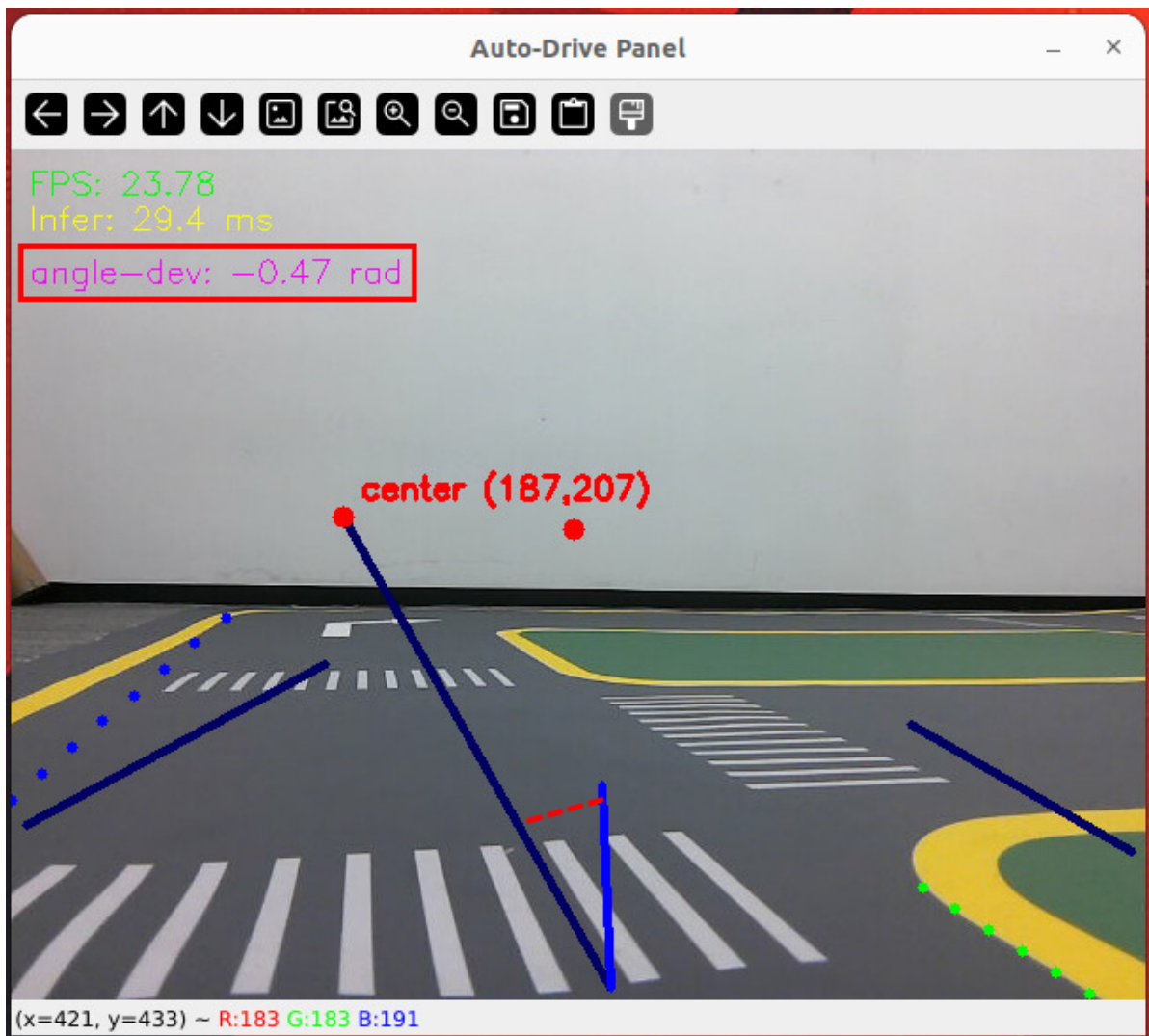
## 5. Lane Straight Keeping—PID Debugging

- When the angle (deviation angle) between the vehicle body centerline and the actual left-right lane intersection point and the vehicle body centerline starting point exceeds the `min_error_angular` threshold, vehicle head direction adjustment will be triggered, thereby keeping the vehicle body driving straight.

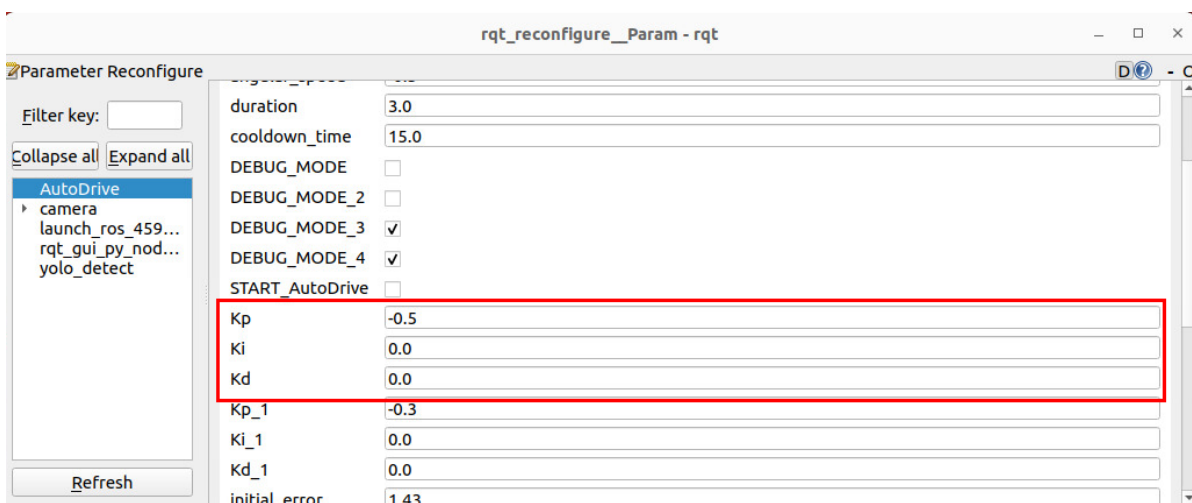
[!WARNING]

- Can be understood as:
- Deviation angle < `min_error_angular` threshold: Perform lane left-right centering keeping
- Deviation angle > `min_error_angular` threshold: Perform lane straight keeping
- Therefore, `min_error_angular` should not be set too large, otherwise it may cause premature entry into left-right translation state, and the vehicle may drift diagonally in the lane.
- The `angle-dev` in the upper left corner of the image below shows the real-time deviation angle





- The vehicle body's left-right keeping in the lane is controlled by PID on the left-right deviation. The control parameters are Kp, Ki, Kd. We can temporarily adjust and observe the actual effect in `rat_reconfigure`



## 5.1 Parameter Adjustment Guide:

[!TIP]

- If the vehicle head adjustment angle is not obvious or sluggish, increase the absolute value of Kp, with a negative sign

- If the vehicle head adjustment angle swings frequently and normal Kp shows overshoot oscillation, you can reduce Kp or introduce Kd differential term to increase damping.
- Ki is used to control steady-state error. Here, in the small-scale scenario of the sandbox map, fast response is needed, so the integral term is not introduced.

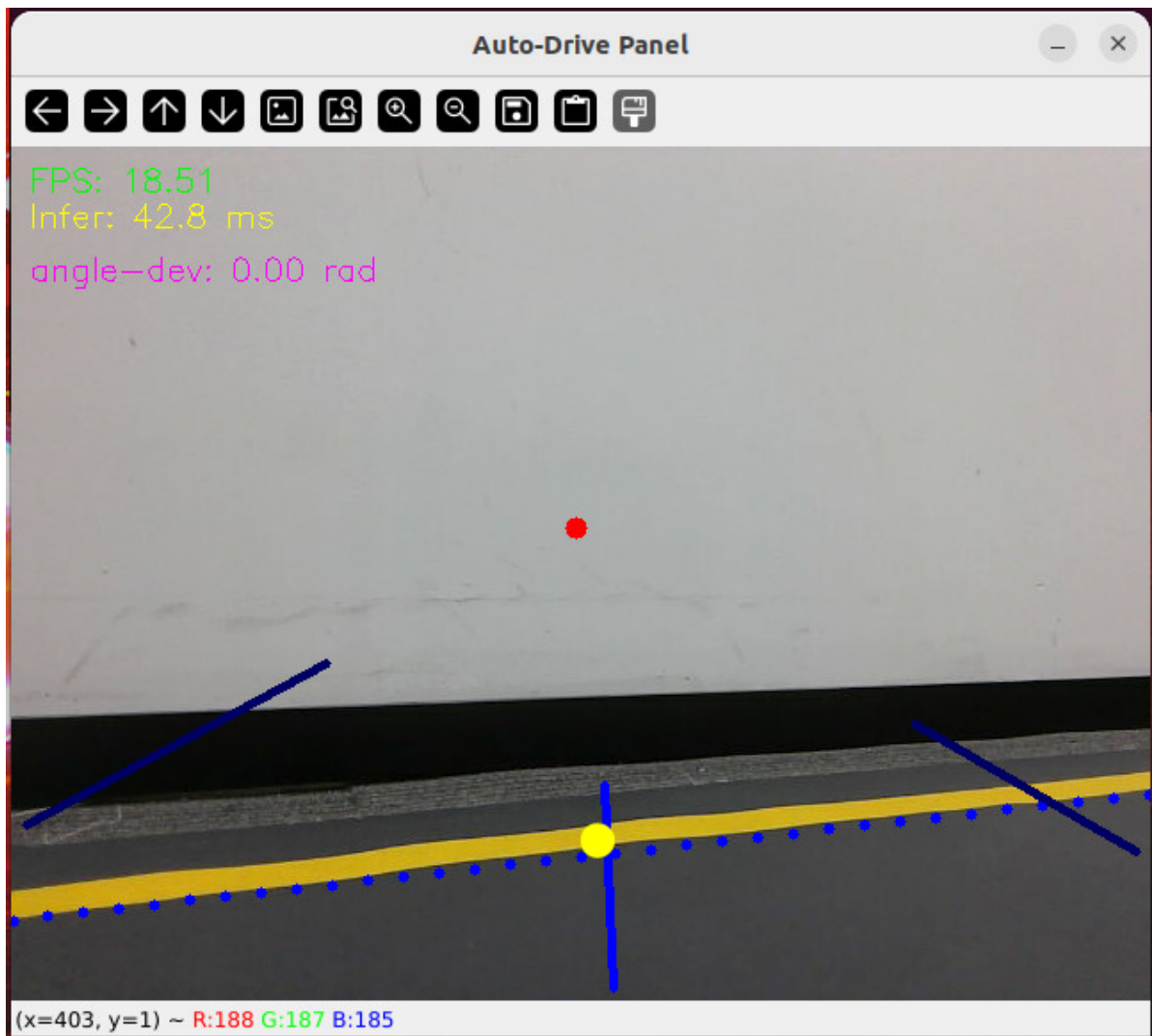
- After parameter debugging is completed, if you need long-term effective parameters, open the configuration parameter file of the auto\_drive node, modify the corresponding parameters, and then save.

```
AutoDrive:
  ros__parameters:
    #Linear_PID parameters
    Kp: -0.5
    Ki: 0.0
    Kd: 0.0
    min_error_angular: 0.03      #Maintain the minimum Angle adjustment
    threshold of the straight line
    max_integral: 1000.0        #Integral limit
```

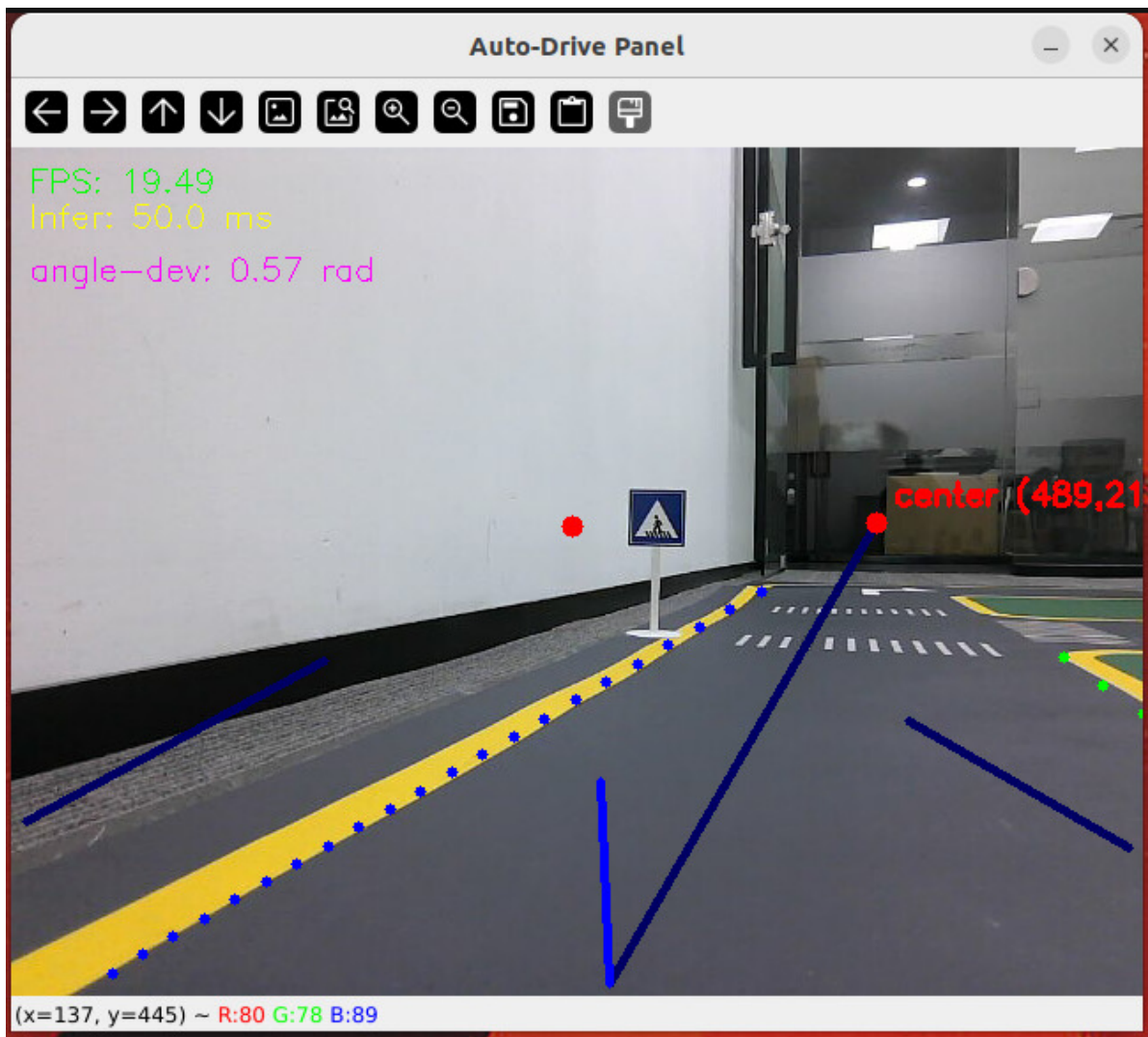
## 6. Curve Turning Effect Debugging

---

- The length of the vehicle body centerline determines the timing of the vehicle entering the curve for turning. When the vehicle body centerline intersects with the left lane line, turning begins.



- When there is no intersection between the vehicle body centerline and the left lane line, the turn is completed and returns to straight keeping mode, correcting direction through the deviation angle.



## 6.1 Parameter Adjustment Guide:

[!TIP]

- If the turning timing is too early, turning before entering the curve indicates that the centerline is too long or the camera perspective is too high. You can appropriately shorten the centerline length or adjust the camera perspective slightly downward.
- Similarly, if the turning timing is too late, turning before entering the curve indicates that the centerline is too short or the camera perspective is too low. You can appropriately extend the centerline length or adjust the camera perspective slightly upward.
- If you adjusted the camera angle, the camera perspective will change, and you need to recalibrate the zero offset error value for lane left-right keeping.

- As mentioned in [07-Centerline Calibration](#), all parameters related to centerline calibration are saved in the `midline_points.yaml` parameter file of the `m3_bringup` package. The file path is:

```
~/yahboomM3_ws/user_ws/src/m3_bringup/config/auto_drive/midline_points.yaml
```

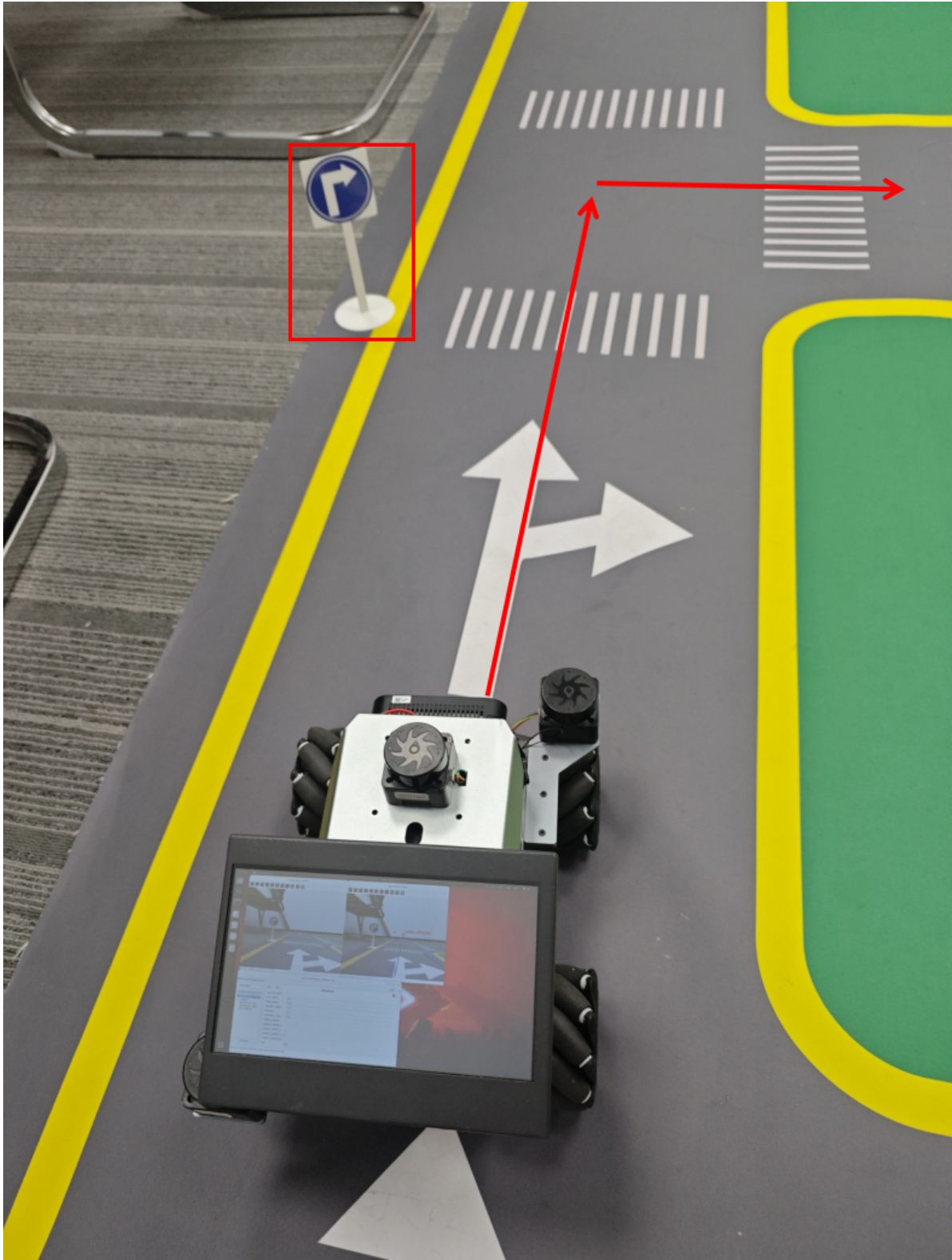
- Among them, `midline_length` is the vehicle body centerline length. It takes effect after modification and saving.



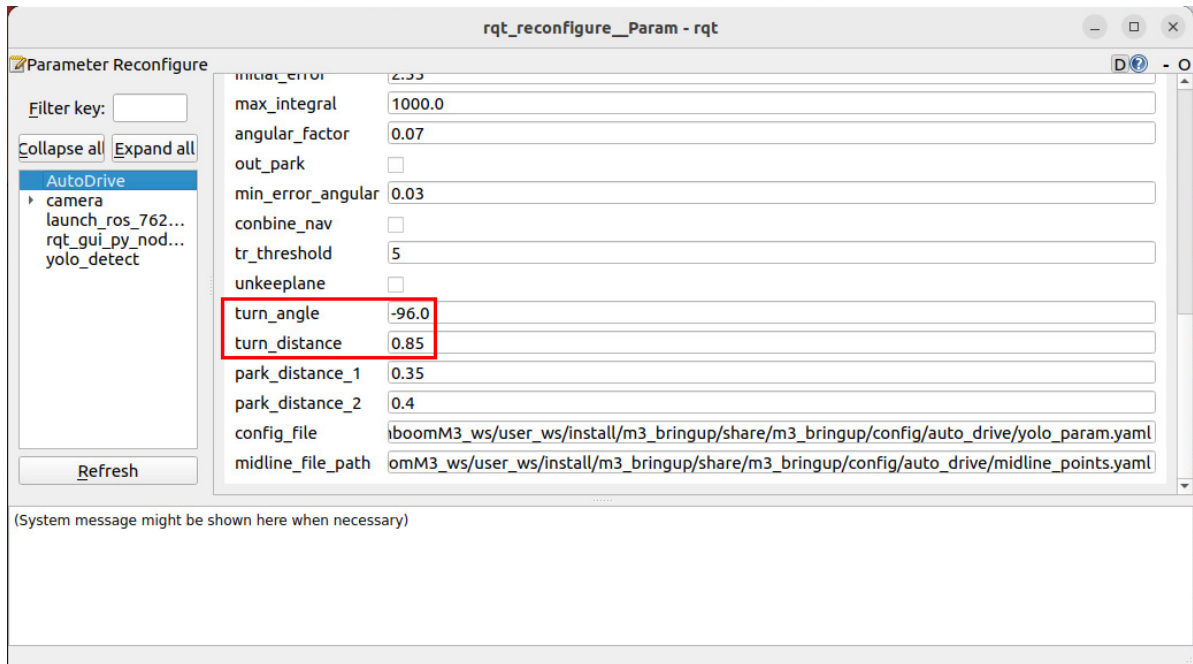
```
midline_points:
-----
midline_length: 115
```

## 7. Right Turn Sign Debugging

- After seeing the right turn sign, the vehicle will drive a certain distance to reach the intersection and then turn right by the specified angle



- Open the `rqt_reconfigure` parameter configuration panel, where:
  - `turn_distance` is the distance to continue driving after seeing the right turn sign, adjusted according to the actual effect
  - `turn_angle` is the angle to turn at the intersection, adjusted according to the actual effect



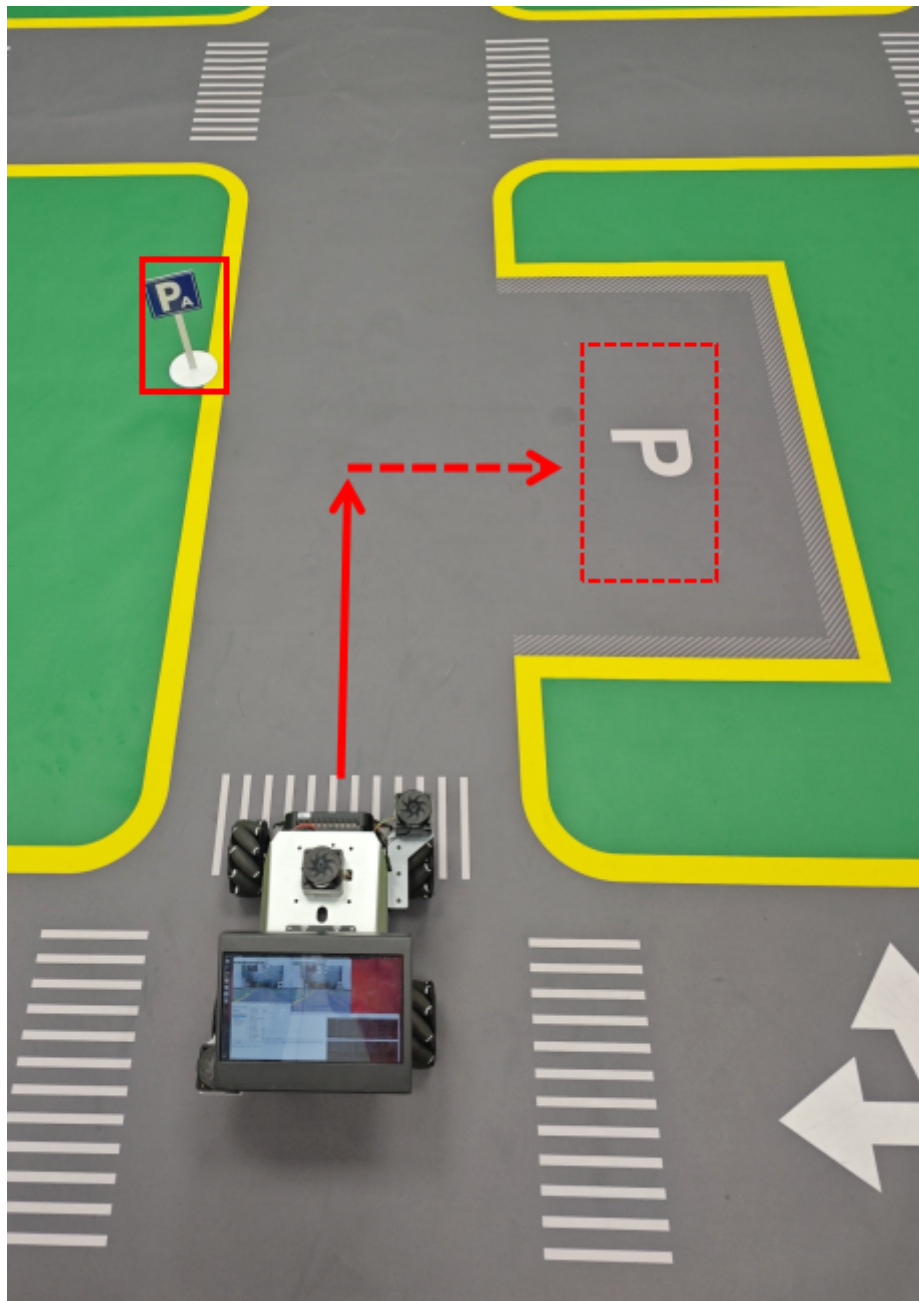
- After parameter debugging is completed, you can fill the parameters into the configuration parameter file of the auto\_drive node, modify the corresponding parameters, and then save.

```
AutoDrive:
  ros__parameters:
    -----
    turn_angle: -96.0          # Turning angle
    turn_distance: 0.85       # Turning distance
    -----
```

## 8. Parking Sign Debugging

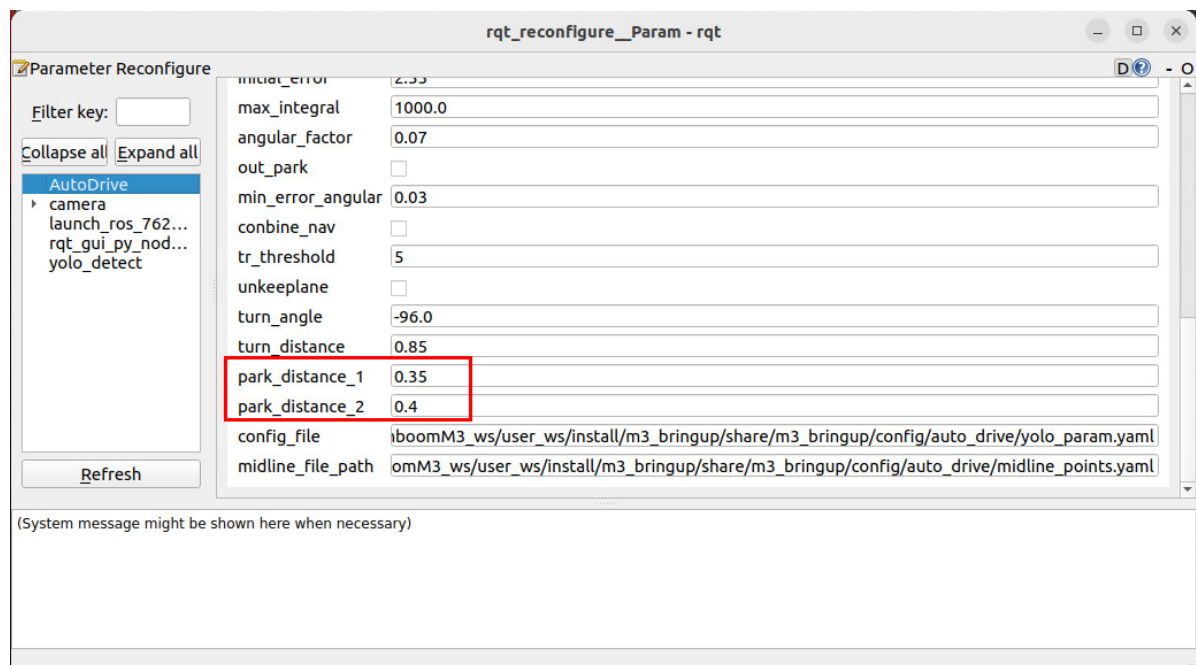
- Autonomous parking means that after seeing the parking sign, the vehicle drives forward a certain distance and then translates into the garage





Open the `rqt_reconfigure` parameter configuration panel, where:

- `park_distance_1` is the distance to continue driving after seeing the parking sign, adjusted according to the actual effect
- `park_distance_2` is the distance to translate right into the garage after continuing driving and stopping, adjusted according to the actual effect



- After parameter debugging is completed, you can fill the parameters into the configuration parameter file of the auto\_drive node, modify the corresponding parameters, and then save.

```
AutoDrive:
  ros__parameters:
    -----
    park_distance_1: 0.35      # Parking distance
    park_distance_2: 0.4      # Parking distance
    -----
```